
Data-Driven Surrogate Modeling for Lid-Driven Cavity Flows

Week 4 Lecture Notes

M&I-ENG 690A – AI in Fluid Mechanics

Summer 2026

Central question. How do we transform qualified CFD fields into fast predictive models without confusing interpolation, data leakage, or a visually plausible field with genuine physical validity?

Instructor: Ehsan Roohi

Contents

1	Where Week 4 Fits in the Course	4
1.1	The complete learning pipeline	4
1.2	How to study this chapter	4
2	The Physical Dataset: Continuum Lid-Driven Cavity	5
2.1	Governing equations and what the network sees	5
2.2	Anatomy of one case file	5
2.3	Why data qualification precedes machine learning	6
2.4	Four distinct error sources	6
3	Three Learning Problems	7
3.1	Parameter-to-scalar surrogate	7
3.2	Coordinate neural field	7
3.3	Operator learning	7
3.4	Function regression versus operator regression	7
3.5	What counts as one sample?	8
4	Train, Validation, and Blind Test	8
4.1	The most dangerous leakage pattern	8
4.2	Visualizing a correct and incorrect split	8
4.3	Training-only model selection	8
4.4	A split protocol for stochastic models	9
5	Nondimensionalization and Normalization	10
5.1	Worked normalization example	10
5.2	Scaling multi-output targets	10
5.3	Normalization is not nondimensionalization	10
6	Baselines Before Neural Networks	11
6.1	Nearest-case baseline	11
6.2	Linear interpolation	11
6.3	Polynomial regression	11
6.4	Why a baseline can outperform a neural model	11
6.5	Baseline decision tree	11
6.6	Minimum baseline reporting standard	12
7	Shallow DNN Surrogate	12
7.1	A compact architecture	12
7.2	Training loop in words	12
7.3	Bias, variance, and seed sensitivity	13
7.4	Learning curves	13
8	Coordinate DNN for Complete Fields	14
8.1	How field data become training rows	14
8.2	Boundary-aware sampling	14
8.3	Local accuracy versus global flow structure	14
8.4	Alternative field representations	14

9 DeepONet: From Functions to Functions	15
9.1 Constructing operator training pairs	15
9.2 Multiple output fields	15
9.3 What we implement in Week 4	16
9.4 Why teach the parameterized version first?	16
9.5 DeepONet training loss	16
9.6 Sensor and coordinate generalization	16
9.7 Failure modes specific to operator models	16
10 Comparing Coordinate DNN and DeepONet	17
10.1 A fair comparison protocol	17
10.2 What not to compare	17
11 Metrics for Fields	17
11.1 Why several metrics are necessary	17
11.2 Physical validation	18
11.3 Computing divergence from a predicted grid	18
11.4 Boundary metrics	18
11.5 Vortex topology	18
12 Interpolation, Extrapolation, and Uncertainty	19
12.1 Three regions of parameter space	19
12.2 Ensemble uncertainty step by step	19
12.3 Aleatoric and epistemic language	19
13 Worked End-to-End Example: Predicting Re=275	19
13.1 Step 1: establish admissibility	19
13.2 Step 2: build the strongest simple prediction	19
13.3 Step 3: query neural models	20
13.4 Step 4: evaluate in layers	20
13.5 Step 5: write a defensible conclusion	20
14 Connection to Week 5: From Data Loss to Physics Loss	21
14.1 Physics-regularized supervised model versus pure PINN	21
14.2 Why Week 4 must come first	21
14.3 Loss competition preview	21
15 Recommended Week-4 Workflow	21
15.1 Reproducibility record	22
16 How to Read the Main Diagnostic Plots	22
16.1 Loss curve	22
16.2 Parity plot	22
16.3 Centerline profiles	22
16.4 Error contour	22
16.5 Streamlines	22
17 Scientific Writing Template for Results	23

18 Common Failure Modes	24
18.1 Diagnostic questions when results look too good	24
18.2 Diagnostic questions when training fails	24
19 Concept Glossary	24
20 Extended Self-Assessment	25
21 Summary	27

1 Where Week 4 Fits in the Course

Week 1 introduced Python, TensorFlow, and a finite-difference lid-driven-cavity solver. Week 2 developed data-driven thinking and neural-network foundations. Week 3 introduced kinetic descriptions, DSMC, particle sampling, and statistical noise. Week 4 now connects these ideas: a numerical solver produces data, and a surrogate learns a map from physical inputs to quantities or fields of interest.

The week deliberately remains *data-driven*. Physics-informed neural networks (PINNs) are introduced in Week 5, after students understand what a supervised surrogate can and cannot do.

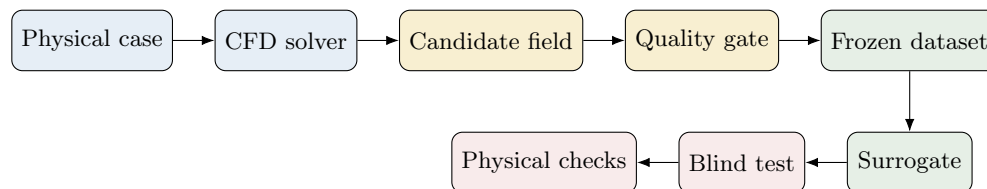
Learning checkpoint

By the end of Week 4 you should be able to:

1. distinguish a scalar surrogate, coordinate neural field, and operator-learning model;
2. build a leakage-free training/validation/test split by flow case;
3. compare interpolation, polynomial regression, and a neural network;
4. evaluate a predicted cavity field using numerical and physical metrics;
5. explain the branch–trunk structure of DeepONet and identify when the word “operator” is justified.

1.1 The complete learning pipeline

The central workflow of the week is shown below. Each arrow represents a scientific decision, not merely a file conversion. A solver configuration produces a candidate field; quality control decides whether the field is admissible; a split defines what generalization means; a baseline establishes the minimum useful performance; and physical validation determines whether a numerical prediction is credible.



1.2 How to study this chapter

This chapter is designed for asynchronous learning. A productive sequence is:

1. read Sections 1–6 before running Lab 1;
2. generate and qualify your assigned CFD case;
3. read Sections 7–10 before running Lab 2;
4. complete the scalar and coordinate-DNN baselines;
5. read Sections 11–13 before running Lab 3;
6. finish with Sections 14–18 and write the scientific interpretation.

At every self-check, attempt an answer before continuing. The goal is not to memorize architectures; it is to learn how to make and defend a predictive claim.

2 The Physical Dataset: Continuum Lid-Driven Cavity

Consider a square cavity of side length L with a lid moving at speed U . The remaining walls are stationary. For an incompressible Newtonian fluid,

$$\text{Re} = \frac{UL}{\nu}. \quad (1)$$

The Week-1 solver returns the nondimensional fields

$$u(x, y; \text{Re}), \quad v(x, y; \text{Re}), \quad \psi(x, y; \text{Re}), \quad \omega(x, y; \text{Re}). \quad (2)$$

The class production grid is fixed at 65×65 so that fields can be stacked without interpolation. A separate grid-sensitivity study may use 97×97 or 129×129 , but those files are not mixed into the production dataset.

2.1 Governing equations and what the network sees

The steady incompressible Navier–Stokes equations in nondimensional form are

$$u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{\partial p}{\partial x} + \frac{1}{\text{Re}} \nabla^2 u, \quad (3)$$

$$u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{\partial p}{\partial y} + \frac{1}{\text{Re}} \nabla^2 v, \quad (4)$$

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \quad (5)$$

The Week-1 streamfunction–vorticity solver does not directly train a neural network and does not directly output pressure. It first computes ψ and ω , then obtains

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}, \quad \nabla^2 \psi = -\omega. \quad (6)$$

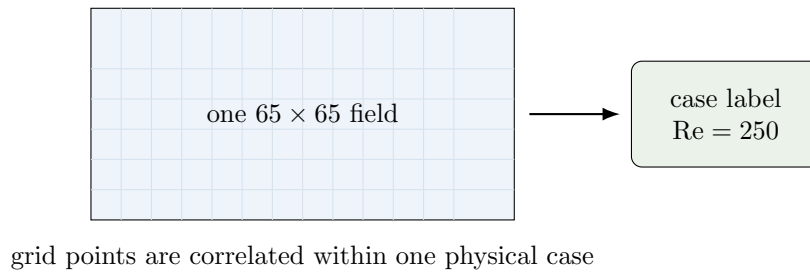
In Week 4, the supervised network sees accepted numerical values of u , v , ψ , or derived observables. It does not “know” the equations above unless they are explicitly placed in a loss function, which is postponed to Week 5.

2.2 Anatomy of one case file

One production file contains arrays and metadata:

Item	Typical shape	Meaning
x, y	$(65,)$ each	nondimensional grid coordinates
u, v, ψ, ω	$(65, 65)$ each	accepted field labels
$\text{Re}, N, \Delta t$	scalars	physical/numerical configuration
steps, residual	scalars	iterative convergence evidence
split, student ID	strings/integers	provenance and permitted use

After merging, the class dataset stores fields with shape $(N_c, 65, 65)$, where N_c is the number of accepted Reynolds-number cases. This first axis is the case axis and must remain intact during splitting.



2.3 Why data qualification precedes machine learning

An ML model approximates its labels, not an inaccessible exact solution. If a CFD field is unconverged or grid-dependent, the network can learn that numerical error with high apparent accuracy. Each field therefore needs a quality card containing:

- final iterative residual and convergence history;
- finite-value and stability checks;
- Ghia centerline errors when a benchmark is available;
- divergence, boundary behavior, vortex location, and vortex strength;
- solver settings and student/case identity.

Important

Iterative convergence is not grid convergence. A residual of 10^{-8} only says that the discrete iteration stopped changing; it does not prove that the discrete solution is close to the continuum solution.

2.4 Four distinct error sources

It is helpful to separate four layers of error:

1. **Modeling error:** incompressible Newtonian Navier–Stokes may not describe a rarefied or compressible experiment.
2. **Discretization error:** finite grid spacing, time step, and derivative approximations alter the PDE solution.
3. **Iterative error:** the discrete equations have not been converged sufficiently.
4. **Surrogate error:** the ML prediction differs from the accepted numerical label.

A small surrogate error addresses only item 4. It cannot repair items 1–3. This separation should appear explicitly in every final report.

Self-check

A neural model predicts the 65×65 CFD labels with 0.2% error, but those labels differ from a grid-converged solution by 8%. Can the model be called an accurate approximation of the continuum flow? Explain which error has actually been measured.

3 Three Learning Problems

3.1 Parameter-to-scalar surrogate

The simplest target is a vector of physically interpretable observables,

$$\text{Re} \mapsto \mathbf{q}(\text{Re}) = [\psi_{\min}, x_v, y_v]. \quad (7)$$

Here (x_v, y_v) is the location of minimum streamfunction and $\psi_{\min}$ measures primary-vortex strength. This problem has few training samples because one CFD case provides one input–output pair.

3.2 Coordinate neural field

A pointwise model uses

$$(\text{Re}, x, y) \mapsto (u, v). \quad (8)$$

Each grid point becomes a training row, but the statistically meaningful unit remains the complete Reynolds-number case. Grid points from one case are correlated and must not be randomly split between training and test sets.

3.3 Operator learning

A mathematical operator maps a function to another function:

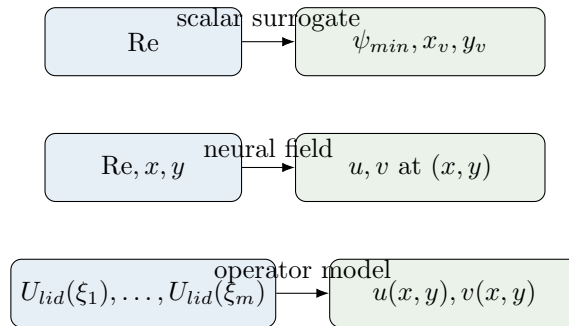
$$\mathcal{G} : a(\xi) \mapsto s(\mathbf{x}). \quad (9)$$

Examples include mapping a boundary velocity profile $U_{lid}(x)$ to a velocity field, or mapping an initial condition to a time-dependent solution. Operator learning differs from ordinary regression because both input and output are functions.

3.4 Function regression versus operator regression

The distinction is easiest to see by asking what must change when a new query is made.

- A scalar surrogate approximates one finite-dimensional function $q(\text{Re})$.
- A coordinate DNN approximates one parameterized function $q(\text{Re}, x, y)$.
- An operator network aims to accept a previously unseen input function and return its output function without retraining.



3.5 What counts as one sample?

For the scalar problem, one Reynolds number gives one sample. For a coordinate model, one can construct many training rows from one field, but these rows share the same physical condition. For true operator learning, one complete input function and its output function constitute one operator sample. Confusing point count with case count leads to exaggerated claims about dataset size.

Learning checkpoint

In this class, “number of training points” and “number of training flows” must be reported separately. A model trained on 30,000 coordinates from eight Reynolds numbers has eight training flows, not 30,000 independent flows.

4 Train, Validation, and Blind Test

For this class, the split is made by Reynolds number. The test cases are hidden from fitting, normalization, architecture selection, early stopping, and loss-weight selection.

Role	Reynolds numbers	Permitted use
Training	100, 150, 200, 225, 250, 300, 350, 400	Fit parameters and scalars
Blind test	175, 275, 375	Final frozen evaluation only

With a small dataset, training cases may be internally assessed using leave-one-case-out cross-validation. A validation case can rotate through the training set. The blind cases remain unopened until the design is frozen.

4.1 The most dangerous leakage pattern

Suppose every 65×65 field is flattened and all 46,475 rows from eleven fields are randomly split. The network then sees points from every Reynolds number during training. A small test error measures spatial interpolation inside already-seen cases, not generalization to a new flow condition.

Self-check

If 80% of the grid points from $Re = 275$ are in training and 20% are in test, is $Re = 275$ a blind test? **No.** The physical case has leaked.

4.2 Visualizing a correct and incorrect split

Incorrect: random grid-point split Correct: complete-case split



4.3 Training-only model selection

The small number of cases makes a single validation split unstable. Leave-one-case-out cross-validation provides a more informative procedure:

1. choose one training Reynolds number as a temporary validation case;

2. fit on the remaining training cases;
3. record the validation error;
4. rotate through all training Reynolds numbers;
5. select degree, width, depth, learning rate, or epoch budget from the mean validation error;
6. retrain on all training cases and evaluate the blind tests once.

This procedure does not create more data, but it uses the available training cases more transparently.

4.4 A split protocol for stochastic models

A stochastic DNN should be evaluated at multiple seeds. Hyperparameters are chosen from training-only cross-validation, while seed variability is reported after the architecture is fixed. Do not search many seeds and publish only the best one. Report mean, standard deviation, and the exact seed set.

Important

Blind test means blind to decisions, not invisible to the person who generated the CFD file. A test-case producer may inspect the CFD quality, but neither the field nor any derived metric can guide ML design.

5 Nondimensionalization and Normalization

Use nondimensional variables before statistical scaling:

$$x^* = x/L, \quad y^* = y/L, \quad u^* = u/U, \quad v^* = v/U. \quad (10)$$

Then compute scaler parameters using training cases only. For example,

$$\widetilde{\text{Re}} = 2 \frac{\text{Re} - \text{Re}_{\min}^{\text{train}}}{\text{Re}_{\max}^{\text{train}} - \text{Re}_{\min}^{\text{train}}} - 1. \quad (11)$$

Coordinates already lie in $[0, 1]$, but mapping them to $[-1, 1]$ can suit tanh activations. Never recompute normalization using blind-test values.

5.1 Worked normalization example

Suppose training Reynolds numbers range from 100 to 400. For $\text{Re} = 250$,

$$\widetilde{\text{Re}} = 2 \frac{250 - 100}{400 - 100} - 1 = 0. \quad (12)$$

For the blind interpolation case $\text{Re} = 175$,

$$\widetilde{\text{Re}} = 2 \frac{175 - 100}{300} - 1 = -0.5. \quad (13)$$

The transformation uses training bounds even when applied to test cases. An extrapolation case such as $\text{Re} = 500$ would produce $\widetilde{\text{Re}} > 1$; the network is then queried outside the normalized domain it saw during training.

5.2 Scaling multi-output targets

The targets $\psi_{\min}$, x_v , and y_v have different numerical ranges. Without output scaling, the largest-magnitude target can dominate mean-square error. A common training-only standardization is

$$\tilde{q}_j = \frac{q_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}}. \quad (14)$$

Predictions are transformed back before physical errors are reported. Store μ_j^{train} and σ_j^{train} with the trained model.

5.3 Normalization is not nondimensionalization

Nondimensionalization is based on physics and produces comparable variables such as u/U . Statistical normalization is based on the observed training distribution and helps optimization. The operations answer different questions and should be documented separately.

Self-check

Why is it leakage to calculate the Reynolds-number mean and standard deviation from all eleven cases, even though no velocity value is used? Because test-case information has already influenced the transformation applied during training.

6 Baselines Before Neural Networks

6.1 Nearest-case baseline

Choose the training field at the nearest Reynolds number. This establishes a minimum performance level.

6.2 Linear interpolation

For $\text{Re}_a < \text{Re}_* < \text{Re}_b$,

$$\mathbf{q}(\text{Re}_*) = (1 - w)\mathbf{q}(\text{Re}_a) + w\mathbf{q}(\text{Re}_b), \quad w = \frac{\text{Re}_* - \text{Re}_a}{\text{Re}_b - \text{Re}_a}. \quad (15)$$

The same formula applies pointwise to complete fields. Because the cavity family is smooth and one-dimensional in parameter space, this is a strong baseline.

Worked example. To estimate $\psi_{\min}$ at $\text{Re}_* = 175$ from training cases 150 and 200,

$$w = \frac{175 - 150}{200 - 150} = 0.5, \quad (16)$$

so the prediction is the arithmetic mean of the two neighboring values. For a full field, the same w multiplies every corresponding grid point. This is valid only because grids and nondimensionalization are identical.

6.3 Polynomial regression

For a scalar target,

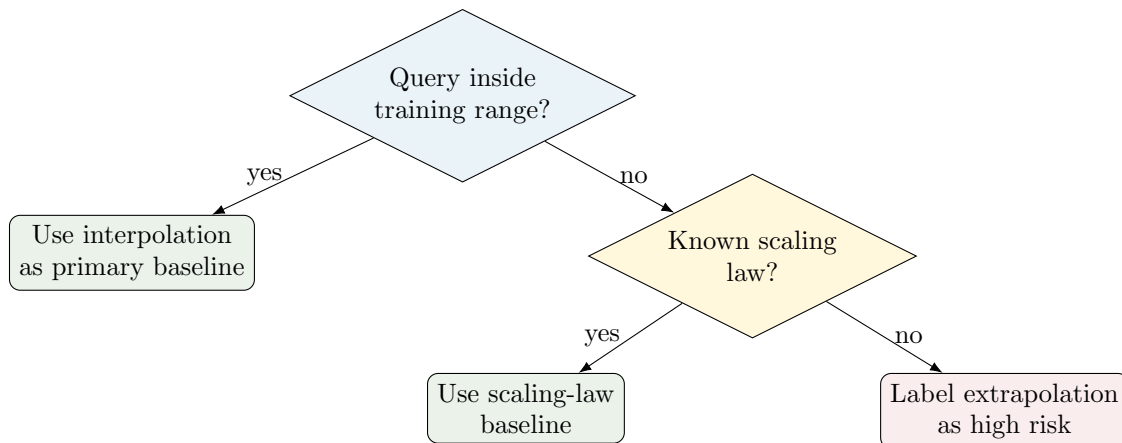
$$q(\text{Re}) = a_0 + a_1\widetilde{\text{Re}} + a_2\widetilde{\text{Re}}^2. \quad (17)$$

A high polynomial degree is dangerous with eight training cases. Complexity must be selected using training-only cross-validation.

6.4 Why a baseline can outperform a neural model

A neural network estimates many weights from very few independent flow cases. Linear interpolation uses the strong prior that the solution changes smoothly between neighboring parameters. When that prior is appropriate, interpolation has low variance and almost no training error mechanism. A DNN becomes more attractive when parameter dimension grows, geometry or boundary functions vary, data are irregular, or many related outputs share structure.

6.5 Baseline decision tree



6.6 Minimum baseline reporting standard

For each blind case, report the two training cases used by interpolation, the interpolation weight, and the resulting error. For polynomial regression, report degree and cross-validation rule. For nearest-case selection, report parameter distance. These details make the baseline reproducible and prevent vague comparisons.

Research-level interpretation

A DNN that fails to beat linear interpolation on blind cases has not demonstrated predictive value. Neural complexity is not itself a scientific contribution.

7 Shallow DNN Surrogate

A scalar DNN takes normalized Reynolds number as input and returns three observables. For layer ℓ ,

$$\mathbf{h}^{(\ell+1)} = \sigma\left(W^{(\ell)}\mathbf{h}^{(\ell)} + \mathbf{b}^{(\ell)}\right). \quad (18)$$

With so few flow cases, a compact model is preferable: two or three hidden layers with 16–64 neurons are adequate for demonstration. Use early stopping and repeat several random seeds.

The supervised loss is

$$\mathcal{L}_{data} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{q}}_i - \mathbf{q}_i\|_2^2. \quad (19)$$

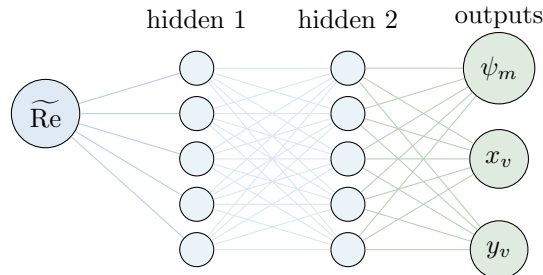
This model is *not physics-informed*. Its labels came from CFD, but its loss does not enforce a governing equation.

7.1 A compact architecture

For the scalar problem, a defensible starting architecture is

$$1 \longrightarrow 32 \longrightarrow 32 \longrightarrow 3, \quad (20)$$

with tanh activations in hidden layers and a linear output. The three outputs are standardized vortex strength and coordinates. The model contains far more adjustable parameters than independent CFD cases, so architecture size alone cannot guarantee generalization.



7.2 Training loop in words

Each epoch performs four operations:

1. pass training Reynolds numbers through the network;
2. compare standardized predictions with standardized targets;

3. differentiate loss with respect to weights using backpropagation;
4. update weights with Adam or another optimizer.

Early stopping monitors a training-only validation signal or a predeclared training-loss rule. A patience value is not chosen after looking at blind-test curves.

7.3 Bias, variance, and seed sensitivity

A tiny network may underfit the trend (high bias); a large network may reproduce eight cases while oscillating between them (high variance). Different random initializations may produce different interpolants even when training loss is similar. This is why at least three seeds are required. Plotting predictions continuously over Re can reveal unphysical oscillations invisible at the discrete case locations.

7.4 Learning curves

A useful diagnostic plots training and cross-validation error against the number of training cases. If both errors are high, model or features are inadequate. If training error is low and validation error remains high, adding capacity is unlikely to help; more independent flows or stronger structure is needed.

Self-check

Two DNNs have identical blind MAE. Model A has 1,200 parameters and small seed variability; Model B has 80,000 parameters and large seed variability. Which is more defensible for eight training flows, and why?

8 Coordinate DNN for Complete Fields

The coordinate model receives $[\widetilde{\text{Re}}, x^*, y^*]$ and predicts $[u^*, v^*]$. It shares one network across all cases and coordinates. Benefits include continuous query locations and modest memory. Limitations include spectral bias, weak treatment of global topology, and possible boundary violations.

To prevent large interior zones from dominating the loss, a zonal form may be used:

$$\mathcal{L} = \lambda_{\text{int}} \mathcal{L}_{\text{int}} + \lambda_{\text{wall}} \mathcal{L}_{\text{wall}} + \lambda_{\text{corner}} \mathcal{L}_{\text{corner}}. \quad (21)$$

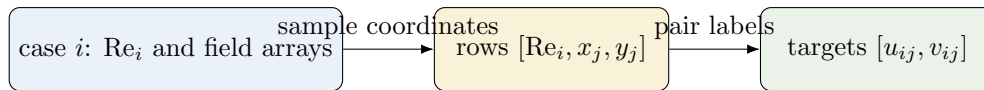
Weights must be chosen without consulting blind tests. Week 5 will show how divergence or PDE residuals can be added as physics penalties.

8.1 How field data become training rows

For one training case Re_i , every selected coordinate produces

$$\mathbf{x}_{ij} = [\widetilde{\text{Re}}_i, \widetilde{x}_j, \widetilde{y}_j], \quad \mathbf{q}_{ij} = [u_{ij}, v_{ij}]. \quad (22)$$

Stacking rows across training cases creates the optimization table. Spatial subsampling reduces cost, but it must preserve walls and high-gradient regions. Uniform stride sampling can accidentally underrepresent the moving lid or corners.



8.2 Boundary-aware sampling

The interior contains roughly $(N-2)^2$ points, while the four walls contain only $O(N)$ points. A uniformly flattened dataset is therefore dominated by the interior. Three remedies are conceptually distinct:

1. oversample walls and corners;
2. assign larger loss weights to wall/corner rows;
3. impose boundary conditions analytically through the network representation.

Week 4 uses the first two only as optional studies. Week 5 discusses physics and hard constraints.

8.3 Local accuracy versus global flow structure

A pointwise MSE treats coordinates independently, yet a vortex is a global topological object. Two fields can have similar MSE but different streamline topology. Consequently, reconstruction must be plotted as a field, not judged solely from a table of pointwise errors.

Important

A coordinate DNN predicts continuous values between grid points, but continuity of the network does not imply conservation of mass or momentum. Smoothness is not the same as physical consistency.

8.4 Alternative field representations

Other reasonable models include POD coefficient regression, convolutional autoencoders, CNN super-resolution, Fourier neural operators, and graph neural networks. The coordinate DNN is selected here because it exposes the role of (x, y) and prepares students for the trunk network in DeepONet.

9 DeepONet: From Functions to Functions

DeepONet contains two subnetworks. The **branch network** encodes samples of an input function,

$$\mathbf{b} = B(a(\xi_1), \dots, a(\xi_m)), \quad (23)$$

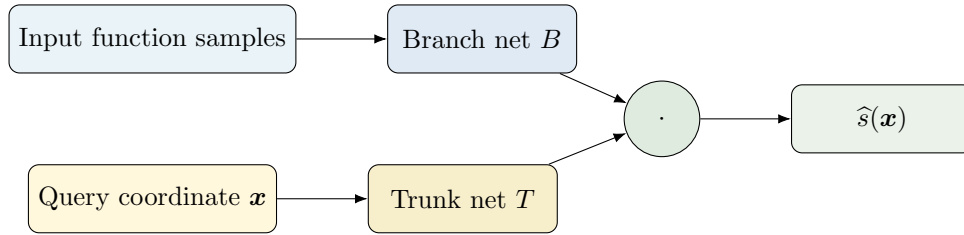
and the **trunk network** encodes an output coordinate,

$$\mathbf{t} = T(\mathbf{x}). \quad (24)$$

The scalar output is commonly written

$$\widehat{s}(\mathbf{x}) = \sum_{k=1}^p b_k t_k + b_0. \quad (25)$$

The latent width p controls how many branch–trunk feature products are available. The original operator-approximation motivation is related to representing an output function through coefficients determined by the input function and basis-like features determined by the query coordinate. In practice, both representations are learned jointly.



9.1 Constructing operator training pairs

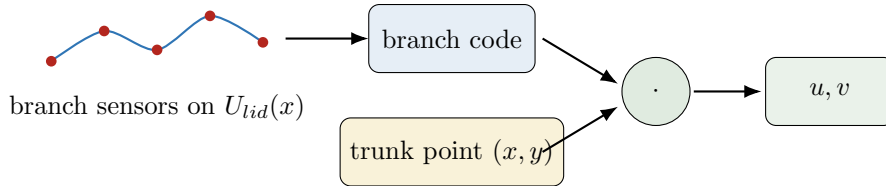
Suppose the input is a lid profile $a(x) = U_{lid}(x)$. Choose m fixed branch sensors $\xi_1, \dots, \xi_m$. For input function a_i , the branch vector is

$$\mathbf{a}_i = [a_i(\xi_1), \dots, a_i(\xi_m)]. \quad (26)$$

For every query coordinate $\mathbf{x}_j = (x_j, y_j)$, the target is $s_i(\mathbf{x}_j)$. Thus one function pair generates many branch–trunk rows:

$$(\mathbf{a}_i, \mathbf{x}_j) \mapsto s_i(\mathbf{x}_j). \quad (27)$$

During inference, a new lid function is sampled at the same branch sensors, encoded once, and queried at as many trunk coordinates as desired.



9.2 Multiple output fields

For velocity, the branch can return separate coefficient sets $\mathbf{b}^u$ and $\mathbf{b}^v$ while the trunk supplies shared spatial features:

$$\widehat{u} = \sum_{k=1}^p b_k^u t_k, \quad \widehat{v} = \sum_{k=1}^p b_k^v t_k. \quad (28)$$

Sharing the trunk encourages both outputs to use a common spatial representation, but it does not enforce incompressibility.

9.3 What we implement in Week 4

The class notebook uses Reynolds number in the branch and (x, y) in the trunk:

$$B(\widetilde{\text{Re}}), \quad T(x, y) \longrightarrow (u, v). \quad (29)$$

This is a **parameterized DeepONet-style architecture**. It is useful for learning a parametric solution family, but it is not a full demonstration of learning an arbitrary input function. A genuine cavity operator-learning dataset could vary the lid profile $U_{lid}(x)$; the branch would receive sensor values $[U_{lid}(\xi_1), \dots, U_{lid}(\xi_m)]$.

Important

Do not claim operator generalization from a dataset that varies only one scalar parameter. The architecture may be DeepONet-like, but the scientific claim is parameter-to-field generalization.

9.4 Why teach the parameterized version first?

The scalar branch lets students isolate the branch–trunk mechanism without first generating dozens or hundreds of boundary functions. It also allows a fair comparison with the coordinate DNN using the same cavity fields. The cost is a narrower claim: the model learns a Reynolds-parameterized solution family.

9.5 DeepONet training loss

For supervised velocity data,

$$\mathcal{L}_{data} = \frac{1}{N_{pairs}} \sum_{i,j} [(\hat{u}_{ij} - u_{ij})^2 + (\hat{v}_{ij} - v_{ij})^2]. \quad (30)$$

Mini-batches contain branch values, trunk coordinates, and output labels. Rows sharing the same branch value belong to the same physical case. Blind splitting therefore remains case-based.

9.6 Sensor and coordinate generalization

Three different generalization questions should not be mixed:

1. **new coordinate:** query (x, y) not used during training for a known case;
2. **new parameter:** predict an unseen Reynolds number inside or outside the training range;
3. **new function:** predict the response to an unseen boundary/initial function.

Lab 3 tests primarily item 2. A full operator experiment would test item 3 using a family of lid profiles.

9.7 Failure modes specific to operator models

DeepONet can fail because branch sensors miss important input-function structure, the training function family is too narrow, trunk coordinates undersample high-gradient zones, or latent width is insufficient. Conversely, a very large model can memorize training functions. Sensor count, function diversity, and case-level tests should therefore accompany parameter-count studies.

Self-check

A DeepONet is trained on 100 Reynolds numbers but every case has the same constant lid profile. Has the model learned the operator from arbitrary lid profiles to velocity fields? No. It has learned a parameterized family conditioned on Reynolds number.

10 Comparing Coordinate DNN and DeepONet

The coordinate DNN mixes parameters and coordinates at its input. DeepONet keeps a separate representation of the case and the query location, then combines latent features. This separation can be beneficial when many functions and coordinates are available, but no architecture is guaranteed to win on a small dataset.

Property	Coordinate DNN	DeepONet-style model
Inputs	(Re, x, y) together	branch: Re ; trunk: (x, y)
Output	(u, v)	latent inner products for (u, v)
Natural strength	simple neural field	separated case/coordinate encoding
Key risk	local fit without topology	overclaiming operator learning
Required baseline	field interpolation	field interpolation and coordinate DNN

10.1 A fair comparison protocol

Use the same training Reynolds numbers, spatial sampling budget, blind cases, optimizer family, and output scaling. Record parameter count, epochs, wall-clock training time, inference time, and seed variability. If one method receives denser wall sampling or more epochs, disclose the difference.

Accuracy alone may not decide the preferred model. A slightly less accurate model may be more stable across seeds, faster to train, or easier to extend to function inputs. Conversely, architectural elegance does not compensate for worse blind performance.

10.2 What not to compare

Do not compare the best of many DeepONet seeds with one coordinate-DNN seed. Do not tune DeepONet on test errors while leaving the baseline frozen. Do not call lower training loss better generalization. The comparison unit is frozen blind performance under a declared protocol.

11 Metrics for Fields

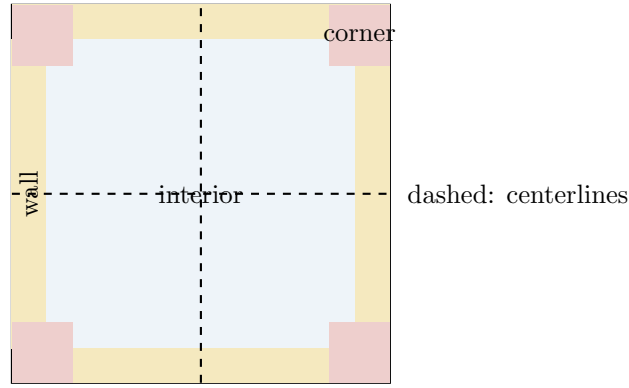
For a predicted velocity field,

$$E_{uv} = \frac{\sqrt{\sum_j [(\hat{u}_j - u_j)^2 + (\hat{v}_j - v_j)^2]}}{\sqrt{\sum_j [u_j^2 + v_j^2]}}. \quad (31)$$

Also report component MAE, maximum vector error, centerline error, wall-zone error, and inference time. Global metrics must be paired with physical checks.

11.1 Why several metrics are necessary

Relative L_2 emphasizes energetic regions and can underweight small secondary vortices. MAE is easier to interpret pointwise but ignores topology. Maximum error identifies extreme local failure but can be controlled by one corner point. Centerline metrics connect to classical benchmarks but observe only two slices. No single metric is sufficient.



Define zones before viewing test errors. A typical wall zone contains points within a fixed distance from a wall; a corner zone is the union of four small squares. Report the full-domain metric even when zonal metrics are emphasized.

11.2 Physical validation

$$E_{div} = \left[\frac{1}{N_{int}} \sum_{int} \left(\frac{\partial \hat{u}}{\partial x} + \frac{\partial \hat{v}}{\partial y} \right)^2 \right]^{1/2}, \quad (32)$$

$$E_{lid} = \frac{1}{N_{lid}} \sum_{lid} |\hat{u} - 1|, \quad (33)$$

$$E_{wall,n} = \frac{1}{N_{wall}} \sum_{wall} |\hat{\mathbf{u}} \cdot \mathbf{n}|. \quad (34)$$

Inspect centerlines, streamlines, vortex displacement, and corner zones. A low global error can coexist with a wrong secondary vortex or severe boundary error.

11.3 Computing divergence from a predicted grid

If the model is evaluated on a uniform grid, centered finite differences approximate interior derivatives. Use one-sided formulas or exclude the boundary when summarizing divergence. State the derivative scheme and grid because the reported divergence includes differentiation error.

11.4 Boundary metrics

For stationary walls, evaluate tangential and normal components separately. For the moving lid, report both $|\hat{u} - 1|$ and $|\hat{v}|$. The two top corners contain a discontinuity in the idealized lid condition; report whether corners are included, excluded, or listed separately.

11.5 Vortex topology

At minimum, compare primary-vortex center and inspect whether predicted streamlines form the correct closed recirculation. For higher Reynolds numbers, secondary corner vortices may be small in area and energy, so their presence or absence should be stated independently of global L_2 .

Research-level interpretation

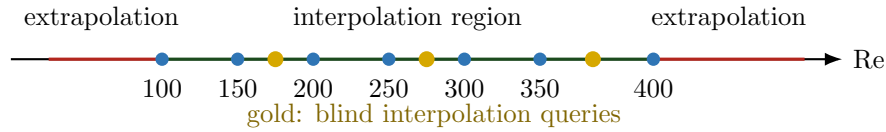
The best model is not the one with the smallest single number. It is the model whose numerical accuracy, physical behavior, uncertainty, and scope of validity are all supported by evidence.

12 Interpolation, Extrapolation, and Uncertainty

Interpolation asks for a Reynolds number inside the training range and bracketed by training cases. Extrapolation asks for a condition outside the observed range. DNNs do not automatically respect scaling laws outside training data. A responsible report must label every test as interpolation or extrapolation.

An ensemble of independently initialized models provides a simple uncertainty indicator. Ensemble spread is not a rigorous confidence interval, but large disagreement can flag difficult regions. Calibration must be tested against actual errors.

12.1 Three regions of parameter space



Interpolation can still fail if the solution changes rapidly between cases or undergoes a bifurcation. Extrapolation can sometimes succeed when a correct scaling structure is learned, but success must be demonstrated rather than assumed.

12.2 Ensemble uncertainty step by step

Train the same architecture with seeds $s_1, \dots, s_M$. At a query point, compute

$$\bar{q} = \frac{1}{M} \sum_{m=1}^M \hat{q}^{(m)}, \quad \sigma_{ens} = \left[\frac{1}{M-1} \sum_{m=1}^M (\hat{q}^{(m)} - \bar{q})^2 \right]^{1/2}. \quad (35)$$

Then test whether σ_{ens} is larger where actual blind error is larger. Low spread can still accompany shared bias, because all ensemble members use the same data and architecture.

12.3 Aleatoric and epistemic language

DSMC sampling noise is an example of data variability that can persist even with a perfect model. Sparse coverage of Reynolds number produces epistemic uncertainty about the learned map. A deterministic CFD dataset has no directly measured label variance, so claims about probabilistic confidence require additional assumptions.

13 Worked End-to-End Example: Predicting Re=275

This example illustrates the reasoning sequence without prescribing the numerical answer.

13.1 Step 1: establish admissibility

The $Re = 275$ CFD field is generated and checked, but labeled as blind test. Its residual and quality card establish that it is a defensible reference; they do not authorize using it for training.

13.2 Step 2: build the strongest simple prediction

For scalar or field interpolation, bracket the query with training cases $Re = 250$ and $Re = 300$. The weight is

$$w = \frac{275 - 250}{300 - 250} = 0.5. \quad (36)$$

This prediction is frozen before the true $Re = 275$ field is compared.

13.3 Step 3: query neural models

The coordinate DNN receives normalized $Re = 275$ at every (x, y) . The DeepONet-style model evaluates the branch once conceptually for the case and the trunk at all grid coordinates. Both reconstruct u and v .

13.4 Step 4: evaluate in layers

1. compare relative L_2 , MAE, and maximum vector error;
2. compare both centerline profiles;
3. compute wall and divergence metrics;
4. inspect error contours and streamlines;
5. compare inference cost and seed variability;
6. decide whether either neural model adds value over interpolation.

13.5 Step 5: write a defensible conclusion

A valid conclusion might be: “The neural model reduced full-field error relative to interpolation but increased lid error, so it improves interior representation without enforcing boundary behavior.” An invalid conclusion is: “Deep learning solved the cavity better,” unless solver accuracy, metrics, and scope support that statement.

Self-check

If DeepONet has lower global L_2 but misses a corner vortex that interpolation captures, which is better? There is no universal answer; report both observations and connect the decision to the quantity of interest.

14 Connection to Week 5: From Data Loss to Physics Loss

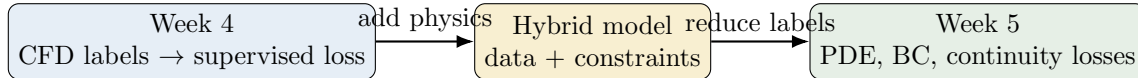
Week 4 trains

$$\mathcal{L} = \mathcal{L}_{data}. \quad (37)$$

Week 5 introduces

$$\mathcal{L} = \lambda_d \mathcal{L}_{data} + \lambda_{bc} \mathcal{L}_{bc} + \lambda_{mom} \mathcal{L}_{momentum} + \lambda_{div} \mathcal{L}_{divergence}. \quad (38)$$

The transition is conceptually important: a supervised model imitates labels; a PINN or physics-regularized model additionally penalizes violation of known equations or constraints. Competing loss components can train at different rates, so loss weighting is a genuine optimization issue, not a cosmetic choice.



14.1 Physics-regularized supervised model versus pure PINN

A physics-regularized supervised model still relies on CFD labels but adds penalties such as divergence or boundary error. A pure forward PINN may use only PDE, boundary, and initial conditions. A hybrid PINN uses both. These labels describe the loss construction, not the network architecture.

14.2 Why Week 4 must come first

Without a strong supervised baseline, a physics-informed model can appear impressive simply because no interpolation or DNN comparison exists. Week 4 establishes data splits, metrics, and physical checks that remain necessary in Week 5. A lower PDE residual does not replace comparison with trusted reference fields.

14.3 Loss competition preview

If $\mathcal{L}_{data}$, $\mathcal{L}_{bc}$, and $\mathcal{L}_{PDE}$ have very different gradient magnitudes, optimizing their weighted sum can prioritize one objective while neglecting another. Week 5 will monitor individual loss terms and physical errors rather than total loss alone.

15 Recommended Week-4 Workflow

1. Run and qualify the common $Re = 100$ CFD case.
2. Generate one assigned production field.
3. Merge only accepted cases and freeze the dataset.
4. Extract scalar vortex observables.
5. Fit interpolation and polynomial baselines.
6. Fit a compact scalar DNN.
7. Build a field-interpolation baseline.
8. Compare coordinate DNN and parameterized DeepONet-style models.
9. Freeze models and evaluate blind cases.
10. Report numerical errors, physical checks, and failures.

15.1 Reproducibility record

For every trained model, record:

- dataset filename and checksum;
- exact training and test Reynolds numbers;
- normalization constants;
- architecture and trainable parameter count;
- optimizer, learning rate, batch size, epochs, and stopping rule;
- random seeds and hardware/runtime;
- training time and inference time;
- filenames of frozen predictions.

This information should be written automatically when possible. A notebook that works only in the original session is not reproducible.

16 How to Read the Main Diagnostic Plots

16.1 Loss curve

A decreasing loss shows optimization progress on the objective, not physical correctness. A plateau can indicate convergence, inadequate learning rate, insufficient capacity, or conflicting objectives. Compare loss scale only between models with identical normalization and loss definitions.

16.2 Parity plot

For scalar targets, predicted versus reference values should lie near the diagonal. With only three blind cases, a parity plot is descriptive, not statistically conclusive. Label every point by Reynolds number.

16.3 Centerline profiles

Centerlines expose oversmoothing, extrema displacement, and wall mismatch. Plot CFD and model on identical axes. Report numerical profile error in addition to visual agreement.

16.4 Error contour

Use a common color scale when comparing models. Plot vector error $\sqrt{(\hat{u} - u)^2 + (\hat{v} - v)^2}$ and, when useful, signed component errors. A concentrated corner error has a different interpretation from a domain-wide bias.

16.5 Streamlines

Streamlines reveal topology but depend on interpolation and plotting density. Use the same grid and plotting settings for reference and prediction. Do not use a visually attractive streamline plot as the only validation.

17 Scientific Writing Template for Results

A compact results paragraph can follow five sentences:

1. state the frozen test and metric;
2. identify the strongest baseline;
3. quantify improvement or degradation;
4. report two physical checks and any conflicting evidence;
5. delimit the validity range and main limitation.

Avoid statements such as “excellent agreement” without numbers. Prefer: “At $Re = 275$, the coordinate DNN reduced relative velocity error from X to Y, but its lid MAE increased from A to B.”

18 Common Failure Modes

Symptom	Likely cause	Required response
Very small test error	Grid-point leakage	Split by complete Reynolds-number case
DNN worse than interpolation	Insufficient data or overfitting	Report baseline honestly; simplify model
Good interior, poor walls	Loss dominated by interior points	Add wall metrics; consider zonal sampling
Wrong streamlines, modest L_2	Metric hides topology	Plot streamlines and vortex location
Excellent training, poor blind test	Memorization	Case-level cross-validation and smaller model
Unstable DeepONet result	Sparse function/case coverage	Reduce claim; repeat seeds; expand data

18.1 Diagnostic questions when results look too good

1. Were complete physical cases separated before flattening?
2. Were normalization statistics fit on training cases only?
3. Was the test case used to choose epochs, loss weights, or seed?
4. Is the reference field accepted by the quality gate?
5. Is the reported metric dominated by near-zero or high-energy regions?
6. Does the model beat linear field interpolation?

Suspiciously small error should trigger an audit before it triggers celebration.

18.2 Diagnostic questions when training fails

Check input/output scaling, finite data, tensor shapes, learning rate, activation saturation, and whether targets were paired with the correct coordinates. Reduce the task to a small subset and verify that the model can overfit one training case; this is a debugging check, not a scientific result. Then restore the declared split and full evaluation.

19 Concept Glossary

Baseline

A simple reference method that a proposed model must outperform or otherwise improve upon.

Blind test

A case excluded from every design decision and used only for final evaluation.

Case-level split

Assignment of complete physical simulations, rather than individual grid points, to train or test sets.

Coordinate neural field

A network that maps coordinates and parameters to local field values.

DeepONet

A branch–trunk architecture designed to approximate nonlinear operators from input functions to output functions.

Epistemic uncertainty

Uncertainty due to limited knowledge, model choice, or sparse coverage; potentially reducible with informative data.

Extrapolation

Prediction outside the covered training domain.

Interpolation

Prediction inside the training domain, usually between observed parameter values.

Operator

A mapping whose input and output are functions.

Quality gate

A declared set of numerical and physical checks required before a simulation enters the dataset.

Surrogate

A fast approximation to a more expensive model, experiment, or input–output relationship.

Zonal metric

An error evaluated in a predefined physical region such as a wall, corner, shock, or vortex zone.

20 Extended Self-Assessment

1. Why can 30,000 grid rows still represent only eight independent training flows?
2. Give one difference between nondimensionalization and standardization.
3. Why is interpolation a particularly strong baseline in this Week-4 dataset?
4. What information enters the branch and trunk in Lab 3?
5. What additional dataset is required to claim function-to-function cavity operator learning?
6. Give an example where lower global L_2 does not mean better physics.
7. Why must loss weights be chosen without blind-test feedback?
8. What is the difference between a physics-regularized supervised model and a pure PINN?
9. Name the four error layers that separate the physical system from the ML prediction.
10. What evidence would justify converting the strongest class project into a journal study?

Brief answer guide

1. Rows within a flow share the same parameter and coherent field structure; the independent case axis has size eight.
2. Nondimensionalization uses physical scales; standardization uses training-set statistics for optimization.
3. The solution family varies smoothly with one scalar parameter and the test points are bracketed.

4. Branch: normalized Reynolds number; trunk: query coordinate (x, y) .
5. Many distinct input functions, such as lid profiles sampled at fixed sensors, paired with their output fields.
6. A model may reduce interior error but violate the lid condition or remove a corner vortex.
7. Otherwise the test set becomes part of model selection and no longer measures generalization.
8. The former uses labels plus physics penalties; the latter may use PDE/BC/IC losses without interior labels.
9. Modeling, discretization, iterative, and surrogate errors.
10. Higher-fidelity data, convergence studies, multiple seeds, stronger baselines, broader parameter coverage, and frozen external tests.

21 Summary

- Data quality is part of the ML method.
- The split must be by flow case, not by grid point.
- Interpolation is a serious baseline for a one-parameter cavity family.
- Numerical error and physical validity are different questions.
- A coordinate DNN learns a neural field; DeepONet separates case and coordinate representations.
- A scalar-parameter branch gives a DeepONet-style parametric surrogate, not full function-to-function operator learning.
- Week 5 adds physics constraints and PINNs to the same cavity problem.

References

1. U. Ghia, K. N. Ghia, and C. T. Shin, “High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method,” *Journal of Computational Physics*, 48, 387–411, 1982.
2. L. Lu, P. Jin, and G. E. Karniadakis, “DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, 3, 218–229, 2021.
3. M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, 378, 686–707, 2019.
4. S. Wang, X. Yu, and P. Perdikaris, “When and why PINNs fail to train: A neural tangent kernel perspective,” *Journal of Computational Physics*, 449, 110768, 2022.